

Lab 3 - Parallelizing k -means, Stat 215A, Fall 2025

October 16, 2025

1 Introduction

This lab investigates the stability of k -means clustering as a method for determining the appropriate number of clusters. The procedure follows the approach introduced by Ben-Hur et al. [1], who proposed using the stability of clustering solutions under data perturbations as a measure of their reliability. In this framework, stability is quantified through the similarity between clusterings obtained from different subsamples of the dataset. A clustering solution is considered stable if repeated runs on perturbed datasets yield highly similar partitions. By analyzing how average similarity varies with k , one can identify the transition from stable to unstable solutions, providing a principled criterion for selecting the number of clusters.

In this lab, the method from [1] is applied to the binary-coded linguistic data from Lab 2, which involves implementing an efficient similarity function between cluster assignments, such as the Jaccard, Matching, or Correlation coefficients, and benchmarking its runtime performance. The stability analysis algorithm is then parallelized to enable efficient computation across multiple values of k and repeated subsampling iterations. Finally, the results are visualized to assess the stability-based criterion for model selection and to evaluate its computational scalability.

2 Similarity Implementation

To compare how similar two clusterings are, three measures were used: the *Jaccard coefficient*, the *Matching coefficient* (or Rand index), and the *Correlation coefficient* (or ϕ coefficient). All three measures are based on comparing the pairwise relationships of data points in two cluster label vectors, $\mathbf{L}_1$ and $\mathbf{L}_2$. For each pair of points (i, j) , it is checked whether they belong to the same cluster in $\mathbf{L}_1$ and in $\mathbf{L}_2$. From these comparisons, four quantities are defined: true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN). The total number of pairs is $T = q(q - 1)/2$, where q is the number of data points [2].

2.1 Jaccard Coefficient

The Jaccard coefficient measures how many pairs of points are placed in the same cluster in both labelings, compared to the number of pairs that are placed together in at least one labeling. It is defined as [1]

$$J = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}. \quad (1)$$

Pairs that are in different clusters in both labelings (TN) are ignored. This measure focuses only on positive agreements and is useful when the goal is to compare the internal structure of clusters rather than overall agreement.

2.2 Matching Coefficient (Rand Index)

The Matching coefficient, also called the Rand index, measures the overall agreement between two clusterings. It counts both pairs that are placed together and pairs that are correctly separated [1]:

$$R = \frac{\text{TP} + \text{TN}}{T}. \quad (2)$$

Because most pairs of points are usually in different clusters, TN often dominates, and the Rand index tends to give higher similarity values. It provides a more global view of agreement between clusterings.

2.3 Correlation Coefficient (ϕ coefficient)

The correlation coefficient, also known as the ϕ coefficient, treats the pairwise co-membership of points as a binary variable and measures the Pearson correlation between the two binary co-membership matrices. It is computed as [1]

$$\phi = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (3)$$

The ϕ coefficient can range from -1 to 1 , where 1 indicates perfect agreement, 0 indicates no correlation, and negative values indicate inverse relationships. This measure is more balanced than the previous two because it normalizes the contingency counts by their marginal totals.

2.4 Choice of similarity measure

For the stability analysis, the Jaccard coefficient was chosen as the primary similarity measure. The Jaccard index quantifies the proportion of point pairs that are clustered together in both partitions, ignoring pairs that are separate in both. This focus on positive co-membership directly reflects the notion of cluster stability, whether data points that belong together remain together under subsampling. In contrast, the Matching (Rand) index includes the large number of true negatives (pairs that are separate in both clusterings), which tends to inflate similarity values and obscure meaningful differences. The correlation (or ϕ) coefficient provides a normalized measure of association but is less intuitive for this purpose. Therefore, following [1], the Jaccard coefficient is used throughout this lab as the most interpretable and relevant indicator of clustering stability.

2.5 Implementation Details

The similarity functions are implemented in `similarity.py`. The function `similarity(labels1, labels2)` takes two one-dimensional arrays of cluster labels corresponding to the same set of data points and returns a single scalar similarity value. Three computational modes were implemented: *normal*, *fast*, and *numba*. Each mode produces identical numerical results but differs in the way pairwise relationships are computed and in its computational cost.

Why co-membership matrices are required Cluster labels themselves carry no intrinsic meaning—the numeric identifiers assigned by algorithms such as k -means are arbitrary and can be permuted between runs. Two clusterings that are identical up to label permutation would appear completely different if the Jaccard index were applied directly to the raw label vectors. The co-membership matrix resolves this by encoding the pairwise relationships among data points, where each entry indicates whether a pair of points belongs to the same cluster. Computing similarity on these binary co-membership matrices therefore compares the structural grouping of the data rather than the arbitrary label numbers [1], [2].

Normal The normal implementation constructs the complete pairwise co-membership matrices

$$C_1[i, j] = (\text{labels1}_i = \text{labels1}_j), \quad C_2[i, j] = (\text{labels2}_i = \text{labels2}_j), \quad (4)$$

and then counts the number of entries corresponding to true positives (TP), false positives (FP), false negatives (FN), and true negatives (TN) [2]. This approach directly follows the theoretical definition of the similarity measures and therefore serves as a reference for correctness. However, it has both time and memory complexity of $\mathcal{O}(q^2)$ and is only feasible for small sample sizes.

To illustrate the *normal* implementation, consider a small example with four data points and two clusterings: $L_1 = [1, 1, 2, 2]$ and $L_2 = [1, 2, 2, 2]$. For each clustering, a binary co-membership matrix C is constructed such that $C[i, j] = 1$ if points i and j belong to the same cluster and $C[i, j] = 0$ otherwise. The resulting matrices are

$$C_1 = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}, \quad C_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}.$$

Each element of the upper triangle of these matrices represents whether the corresponding pair of points is co-clustered in that labeling. The pairwise comparison of C_1 and C_2 for all unordered pairs (i, j) with $i < j$ is shown in Table 1.

Table 1: Pairwise comparison of co-membership matrices C_1 and C_2 for the example with $L_1 = [1, 1, 2, 2]$ and $L_2 = [1, 2, 2, 2]$.

Pair (i, j)	$C_1[i, j]$	$C_2[i, j]$	Category
(1,2)	1	0	FP (same in L_1 , different in L_2)
(1,3)	0	0	TN (different in both)
(1,4)	0	0	TN (different in both)
(2,3)	0	1	FN (different in L_1 , same in L_2)
(2,4)	0	1	FN (different in L_1 , same in L_2)
(3,4)	1	1	TP (same in both)

From the table, one obtains the contingency counts $TP = 1$, $FP = 1$, $FN = 2$, and $TN = 2$, with a total of $T = 6$ unordered pairs. The Jaccard coefficient is then computed as

$$J = \frac{TP}{TP + FP + FN} = \frac{1}{1 + 1 + 2} = 0.25.$$

In practice, the *normal* implementation performs these steps using NumPy broadcasting: it constructs the full boolean co-membership matrices C_1 and C_2 , compares them elementwise to count TP, FP, and FN, and then calculates J from these counts. While conceptually simple, this approach requires $\mathcal{O}(q^2)$ time and memory because the $q \times q$ matrices must be stored in memory, making it infeasible for large q .

Numba The *numba* mode provides a compiled baseline for the direct $\mathcal{O}(q^2)$ algorithm. It performs explicit nested loops over all pairs of data points but is accelerated using the `@jit` decorator from the Numba just-in-time compiler, which translates the Python code into optimized machine instructions at runtime. In contrast to the *normal* implementation, which constructs two dense $q \times q$ co-membership matrices in memory, the Numba version iterates directly over the one-dimensional label arrays. As a result, it avoids allocating gigabyte-scale boolean matrices and eliminates the creation of temporary arrays for operations such as logical `&` and `~`. This drastically reduces the memory footprint from $\mathcal{O}(q^2)$ to $\mathcal{O}(1)$ additional storage.

When the broadcast-based *normal* method is executed, large contiguous blocks of memory are filled with boolean values, which are repeatedly read, written, and combined during the pairwise comparisons. This leads to high memory traffic and frequent cache misses, causing the CPU to wait for data to be fetched from RAM. The Numba implementation, on the other hand, keeps only a few integer counters (TP, FP, FN) in CPU registers and streams sequentially through the label vectors, resulting in good cache locality. Although the algorithmic complexity remains $\mathcal{O}(q^2)$ - since every pair of points is still compared once, the compiled loop executes at native speed without Python interpreter overhead. Because Numba compiles the function to low-level machine code, the loop body runs directly on the CPU, storing intermediate results in registers rather than allocating new Python or NumPy objects in memory. This makes the *numba* implementation a low-level, allocation-free reference that matches the logical behavior of the *normal* method while running orders of magnitude faster and using only a fraction of the memory.

Fast The fast mode was developed to overcome these computational limitations quadratic runtime by avoiding explicit pairwise comparisons. Instead of checking all $\frac{q(q-1)}{2}$ pairs, the algorithm exploits the fact that the number of co-clustered pairs in a partition depends only on the cluster sizes. If a cluster c in a labeling contains n_c data points, the number of point pairs that are co-clustered within that cluster is given by the combinatorial term $n_c(n_c - 1)/2$. Summing this quantity across all clusters yields the total number of pairs that are co-clustered in that labeling:

$$\text{Pairs}(L) = \sum_c \frac{n_c(n_c - 1)}{2}, \quad (5)$$

To compute the intersection between two clusterings L_1 and L_2 , each data point is associated with a pair of labels $(L_1[i], L_2[i])$. By counting how often each label pair occurs, one obtains a two-dimensional contingency table representing the joint distribution of cluster memberships. The number of pairs that are co-clustered in both clusterings corresponds to the same combinatorial count applied to each cell of this joint table:

$$\text{Intersection}(L_1, L_2) = \sum_{a,b} \frac{n_{a,b}(n_{a,b} - 1)}{2}, \quad (6)$$

where $n_{a,b}$ is the number of points that simultaneously belong to cluster a in L_1 and cluster b in L_2 .

For illustration, consider again the clusterings $L_1 = [1, 1, 2, 2]$ and $L_2 = [1, 2, 2, 2]$ from the previous example. Each clustering contains two clusters of sizes $n_{L_1} = (2, 2)$ and $n_{L_2} = (1, 3)$. The total number of within-cluster pairs is therefore

$$\text{pairs}(L_1) = \frac{2(2-1)}{2} + \frac{2(2-1)}{2} = 2, \quad \text{pairs}(L_2) = \frac{1(1-1)}{2} + \frac{3(3-1)}{2} = 3.$$

Next, the joint label pairs (L_1, L_2) are

$$(1, 1), (1, 2), (2, 2), (2, 2),$$

which define three joint clusters with counts $n_{(1,1)} = 1$, $n_{(1,2)} = 1$, and $n_{(2,2)} = 2$. Only the last group contains more than one element, yielding $\text{TP} = \frac{2(2-1)}{2} = 1$. From these counts it follows that $\text{FP} = 2 - 1 = 1$, $\text{FN} = 3 - 1 = 2$, and $T = 4(4 - 1)/2 = 6$. Thus, the Jaccard similarity is

$$J = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} = \frac{1}{1 + 1 + 2} = 0.25,$$

which matches the result obtained from the full co-membership matrices in the *normal* implementation. This demonstrates that the *fast* method produces identical outcomes using only simple counting operations and linear-time complexity.

In practice, this computation is implemented efficiently using the function `np.bincount` in NumPy. Each joint label pair (a, b) is mapped to a unique integer code, and the counts $n_{a,b}$ are obtained in a single vectorized operation. This eliminates all Python loops and reduces the complexity from $\mathcal{O}(q^2)$ to approximately $\mathcal{O}(q)$, as each data point contributes to exactly one count. Memory usage is also drastically reduced, since only frequency vectors of size proportional to the number of clusters are required rather than full pairwise matrices. The resulting method is therefore suitable for the large-scale computations required in the stability analysis of k -means.

Numba was not applied to the fast method because the implementation is already fully vectorized with NumPy functions, which are internally optimized C routines. Adding Numba around these operations introduces extra compilation overhead and function call latency, resulting in slightly slower runtimes compared to the pure NumPy version.

Cython was also not used in this because it requires an additional compilation step and build setup. Although Cython can offer advantages for highly optimized, low-level code such as hand-tuned loops with OpenMP parallelization or custom memory management, the existing implementations already achieve near C performance using NumPy's precompiled implementations.

2.6 Algorithmic Efficiency

Table 2 reports runtimes (in seconds) for the three implementations on two input sizes, $q = 5,000$ and $q = 29,000$. The *normal* mode materializes full $q \times q$ co-membership matrices; the *numba* mode runs an allocation-free $\mathcal{O}(q^2)$ double loop; the *fast* mode uses frequency counting (joint histogram) to avoid pairwise enumeration.

Scaling behavior The *normal* and *numba* modes exhibit the expected $\mathcal{O}(q^2)$ growth: the ratio $(29,000/5,000)^2 \approx 33.6$. Empirically, *numba* scales by $\approx 36.5\times$ (from 0.00757 to 0.27584 s), close to the quadratic prediction,

Table 2: Runtime (seconds) of similarity computation by mode and problem size.

Mode	$q = 5,000$	$q = 29,000$
Normal	0.0998	4.9053
Numba	0.0076	0.2758
Fast	0.0002	0.0012

while *normal* scales by $\approx 49\times$, reflecting additional memory-allocation and bandwidth costs from constructing dense $q \times q$ boolean matrices. In contrast, the *fast* mode increases by only $\approx 7.3\times$ (from 0.000170 to 0.001235 s) as q grows by $5.8\times$, which is consistent with near-linear $\mathcal{O}(q)$ behavior plus small fixed overheads.

Speedups At $q = 29,000$, *numba* is $\approx 18\times$ faster than *normal* ($4.905/0.276$), and *fast* is $\approx 4,000\times$ faster than *normal* ($4.905/0.001235$). At $q = 5,000$, *numba* is $\approx 13\times$ faster and *fast* is $\approx 590\times$ faster than *normal*. These results confirm that avoiding explicit pairwise enumeration yields orders-of-magnitude improvements.

Comparison to the lab instructions The instructions note an $\mathcal{O}(q^2)$ Jaccard implementation running in ~ 0.32 s for $q = 29,000$ and a “smarter” implementation around $58\mu\text{s}$. The measured *numba* result (0.276 s) aligns well with the cited $\mathcal{O}(q^2)$ reference. The *normal* result (4.905 s) is slower due to constructing large dense matrices (multi-GB working set) and the attendant memory traffic. The *fast* result (1.24 ms) is substantially faster than the quadratic baselines and within the same qualitative regime as the “smarter” approach, though still above the quoted $58\mu\text{s}$. However, the empirical behavior matches the theoretical expectations and the guidance given in the lab handout: *fast* is the appropriate mode for large q , *numba* is a strong $\mathcal{O}(q^2)$ baseline without large allocations, and *normal* primarily serves as a correctness reference on small inputs.

2.7 Runtime on the `lingBinary.Rdata` dataset

To evaluate how the similarity computation scales on real data, the function was applied to the binary question responses in `lingBinary.Rdata`, using the columns starting at `Q050_1`. Each Q -column was treated as a binary label vector, and random pairs of such columns were used to test the Jaccard similarity. For each subsampling fraction $m = 0.2 \dots 0.8$, two independent subsamples were drawn, their intersection was determined, and the similarity was computed for *normal*, *numba*, and *fast*. Figure 1 shows the measured runtimes as a function of m , while Table 3 summarizes the numerical results for representative values of m .

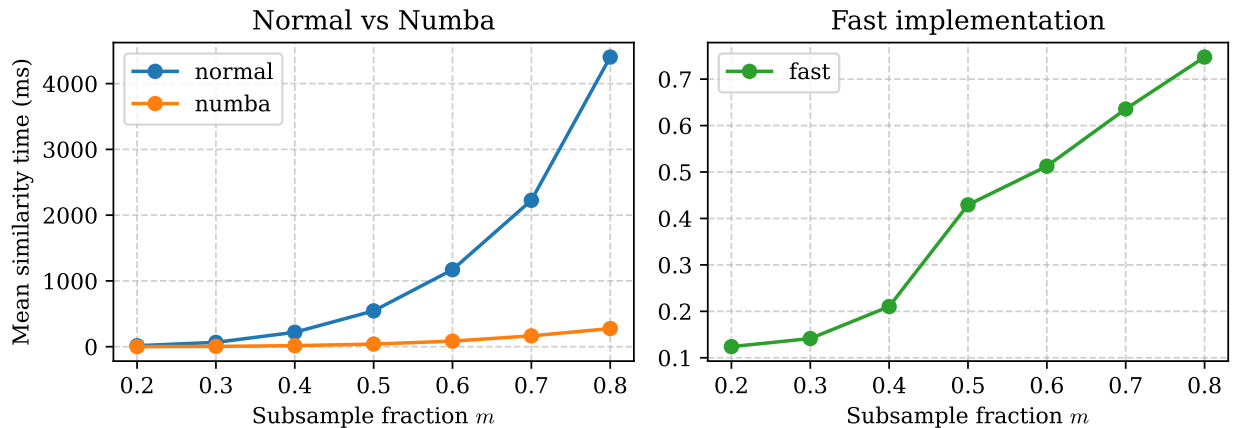


Figure 1: Mean runtime of the similarity computation for different subsample fractions m on the `lingBinary.Rdata` dataset. The left figure compares the *normal* and *numba* implementations (both $\mathcal{O}(q^2)$), while the right panel shows the *fast* implementation, which operates in approximately $\mathcal{O}(q)$.

Table 3: Mean similarity runtimes (ms) for selected m values on `lingBinary.Rdata`.

m	Fast	Numba	Normal
0.2	0.1	1.1	15.3
0.5	0.4	40.0	545.0
0.8	0.7	276.1	4403.8

The results demonstrate the expected scaling behavior of the three implementations. The *fast* version remains below 1 ms across all tested m values, exhibiting near-linear growth consistent with the $\mathcal{O}(q)$ complexity of the counting-based algorithm. In contrast, both the *numba* and *normal* modes show quadratic growth in runtime as the intersection size increases. While the Numba-accelerated implementation eliminates Python overhead and is roughly an order of magnitude faster than the pure NumPy approach, its $\mathcal{O}(q^2)$ complexity still makes it impractical for large datasets. The normal implementation incurs substantial additional overhead from allocating dense $q \times q$ boolean matrices, resulting in multi-second runtimes for $m \geq 0.7$.

3 Algorithm 1 implementation

3.1

Overview Algorithm 1 was implemented in `code/algorithm1.py` as a function `run_algorithm1(df, params)` that computes the stability matrix $S \in R^{B \times (k_{\max}-1)}$ for $k = 2, \dots, k_{\max}$. For each k , the procedure repeats B trials. In each trial, two without-replacement subsamples of fraction m are drawn from the rows of the binary feature matrix. k -means is fitted separately on each subsample to produce two labelings. The similarity is then computed only on the intersection of the original indices (points that appear in both subsamples), using the Jaccard coefficient from `similarity.py` (fast mode by default). The resulting similarity is stored in $S[i, k]$.

3.2 Subsampling and alignment

For each k , two random subsamples are drawn from the dataset, each containing a fraction m of all data points. This means that only about $m \cdot n$ points are used in each clustering run. Because the two subsamples are drawn independently, some points will appear in both, while others will appear in only one. The points that occur in both subsamples form the intersection, which is the only set of points that can be directly compared between the two clusterings. To make this comparison possible, the cluster labels of both subsamples are aligned according to the order of the intersection indices. If the intersection is too small (fewer than two points), the similarity for that trial is skipped.

3.3 Similarity computation

Once the two label vectors are aligned on the intersection, the similarity between the two clusterings is calculated using the Jaccard index. The implementation uses the fast, vectorized method from `similarity.py`, which counts how many pairs of points are clustered together in both subsamples without building large matrices. This makes the computation much faster and memory-efficient while giving the same result as the standard Jaccard formula, cf. Eq. 1.

3.4 Choice of k -means parameters

The `scikit-learn` implementation (`sklearn.cluster.KMeans`) was used. Key parameters and motivations:

- **n_init**: number of random centroid initializations per fit. A larger value reduces sensitivity to poor initializations. Values of 10–20 are standard; here `n_init = 20` in the final config to stabilize per-trial variability.
- **max_iter**: iteration cap for Lloyd/Elkan updates. Increased to `max_iter = 500` to avoid premature termination on some binary partitions.
- **random_state**: fixed base seed to ensure reproducibility across trials and platforms. The final config uses `random_state = 215`.
- **algorithm**: "elkan" (see below) to accelerate Euclidean k -means via triangle-inequality pruning on dense data; appropriate for our binary-but-dense matrix.

Environment variables (`OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, `MKL_NUM_THREADS`) are set to 1 in the quick test to avoid thread oversubscription when Joblib parallelizes trials.

3.5 Parallelization

The implementation parallelizes the inner loop over trials $i = 1, \dots, B$ for each fixed k via `joblib.Parallel(..., prefer="processes")`. This choice has two advantages: First, the load balancing is simpler because trial runtimes vary (different subsamples/intersections), and second, the memory pressure is lower than parallelizing the outer k loop (which would duplicate per- k resources and complicate aggregation). Moreover, $k_{\max}$ is typically small while B is comparatively large, so parallelizing trials is reasonable. The outer loop over k remains serial.

3.6 Final configuration and rationale

Different parameter configurations have been tested and the final configuration used for reporting is:

```
k_max = 10,  B = 200,  m = 0.70,  n_jobs = -1,  random_state = 215

n_init = 20,  max_iter = 500,  algorithm = "elkan".
```

- $k_{\max} = 10$: maximum number of clusters. Recommended range while keeping compute bounded.
- $B = 200$: number of trials (independent subsample pairs) used to estimate the mean stability $S(k)$. A higher B improves the precision of the estimated stability curve by averaging out random variability between subsamples, particularly at larger subsample fractions m .
- $m = 0.70$: subsample fraction, i.e., the proportion of the dataset drawn without replacement in each subsampling iteration. Larger m values increase the overlap between subsamples, yielding more reliable similarity estimates, while smaller m values reduce computation time. A value of $m = 0.7$ provides a good balance between statistical stability and efficiency and was found to be practical using the fast similarity computation.
- $n_{\text{init}} = 20$ and $\text{max_iter} = 500$: parameters of the k -means clustering algorithm. Each clustering is initialized 20 times with random centroids, and the best solution (smallest within-cluster sum of squares) is retained. Increasing the number of initializations reduces sensitivity to random starts, and allowing up to 500 iterations ensures convergence and lowers thereby the variance of per-trial similarity values.
- $n_{\text{jobs}} = -1$: instructs the computation to use all available CPU cores for the inner parallel loop.
- $\text{random_state} = 215$: fixes the random number generator seed to ensure that the computations are reproducible across runs and platforms.

3.7 K-means Algorithm

Both algorithms minimize the same k -means objective with Euclidean distance. Lloyd’s algorithm performs full distance computations from each point to each centroid at each iteration. Elkan’s algorithm tightens upper/lower bounds on point , i.e. centroid distances and uses the triangle inequality to skip many distance evaluations once bounds certify that a centroid cannot be the nearest. On dense data and moderate/large k , this typically yields substantial speedups without changing the final solution. Our feature matrix is dense (binary 0/1 but stored densely), so Elkan is supported and efficient; for sparse matrices, Elkan is not applicable. Empirically, Elkan reduced per-trial fit times relative to Lloyd on this dataset, justifying this choice.

3.8 Computational setup

All experiments were initially run on a local Apple MacBook Air M4 processor (8 CPU cores, 24 GB unified memory). The stability analysis (Algorithm 1) was parallelized across all available CPU cores using `n_jobs = -1` in `joblib`. Several configurations of parameters were tested, including a very fast, medium, and the final high-precision setting, cf. 3.6. The fast configuration was benchmarked both on the SCF cluster (Gandalf) and on the local machine, and the results showed that the MacBook Air performed comparably and even slightly faster than the SC. For that reason, the final simulations were done locally on the MacBook. The configuration (see Section 3.6 and shown in Figures 2, 3, and 4) used the most precise parameter set and took approximately 4 hours and 55 minutes in total. Each individual iteration took between 1.4 s (for

$k = 2$) and 17.5 s (for $k = 10$). The use of all cores provided near-optimal utilization while keeping memory demands low. It should be noted that the chosen configuration is likely on the conservative and overpowered side. Fewer trials or shorter iteration limits would probably yield similarly reliable results in a fraction of the time, but the settings used to create the figures in this report.

4 Result

After implementing the similarity functions and integrating them into the stability estimation algorithm of Ben-Hur *et al.* [1], the next step was to apply the method to the `lingBinary.Rdata` dataset. Using the efficient *fast* similarity computation, the algorithm was run for a range of cluster numbers k and repeated subsampling iterations. For each k , the stability of the resulting cluster assignments was estimated as the average Jaccard similarity between pairs of subsampled clusterings. This procedure yields an empirical distribution of stability values for every k , from which both summary statistics (mean or median stability) and distributional plots can be derived. The following section presents and discusses these results, examining how stability evolves with increasing k and how an appropriate number of clusters can be selected based on these patterns.

Figure 2 presents the empirical distributions of Jaccard similarities across repeated subsampling runs for several values of k , and Figure 3 shows the corresponding cumulative distributions. In both views the same pattern emerges: for small k the similarity distribution is concentrated at high values (right-shifted CDF), while increasing k shifts mass to lower similarities and increases spread. This is the expected behavior for stability under subsampling [1]: as partitions become finer, co-membership of pairs is harder to reproduce across draws, so the similarity decreases and variance grows.

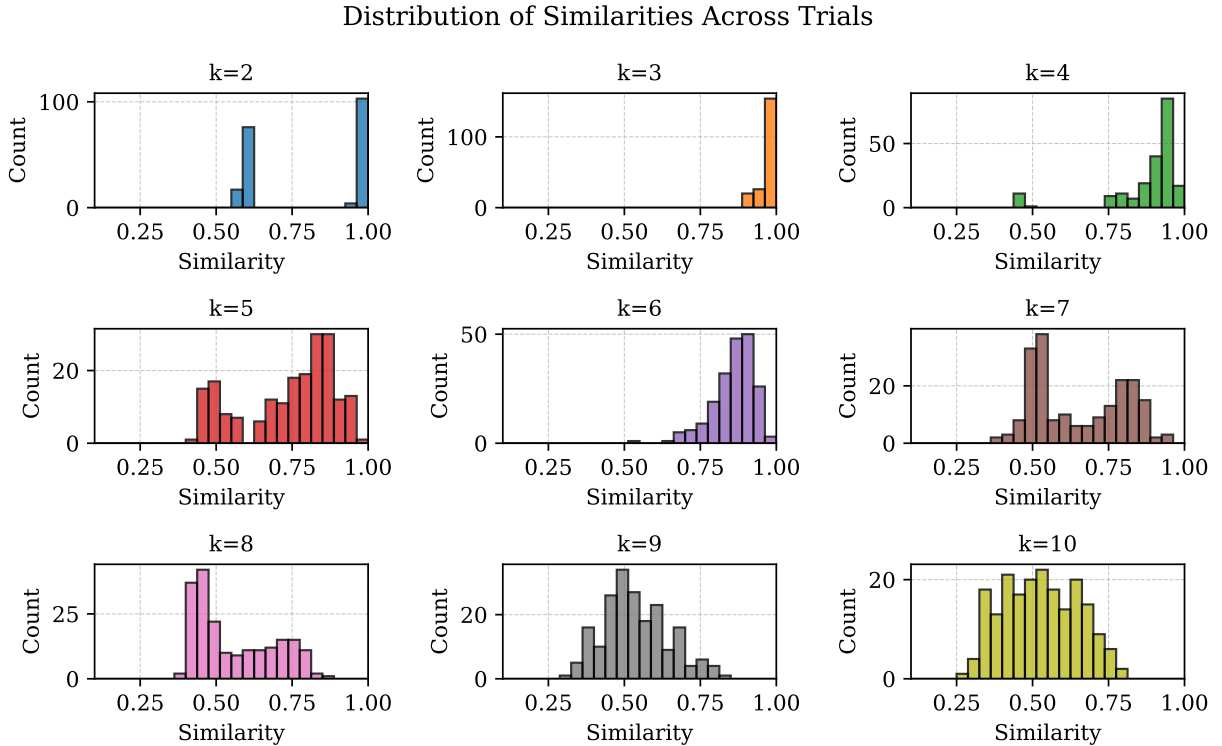


Figure 2: Distributions of Jaccard similarities across subsampling iterations for multiple k . Panels illustrate the progressive shift to lower stability and larger dispersion as k increases.

Figure 4 summarizes the results by plotting stability versus k in a boxplot. The curve shows the typical pattern described by Ben-Hur *et al.* [1]: a high-stability region at small k , followed by a deterioration as k grows and clusters fragment.

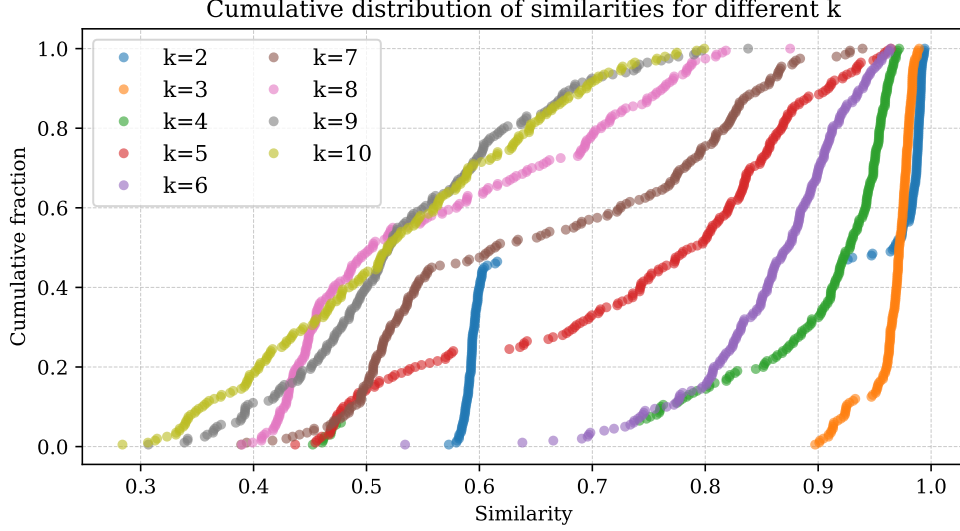


Figure 3: Cumulative distribution functions (CDFs) of Jaccard similarity for selected k . Smaller k yield right-shifted CDFs (high stability), whereas larger k move left, indicating reduced reproducibility under subsampling.

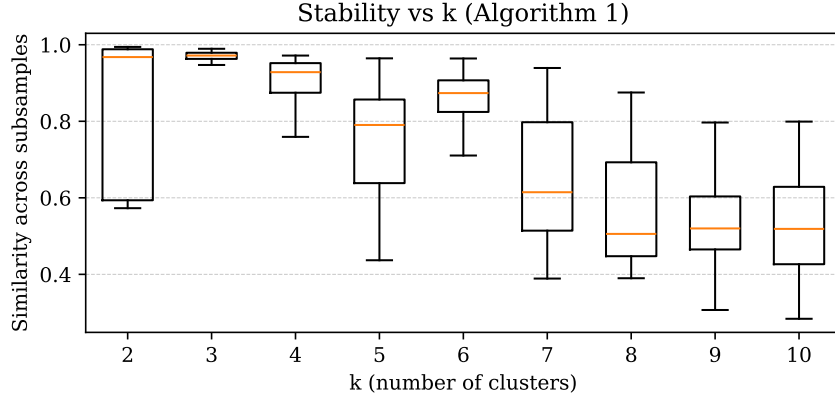


Figure 4: Jaccard Stability across iterations as a function of k . A high, flat region at small k is followed by a marked decline as k increases, consistent with stability-based model selection.

Bimodal Distributions The distributions for $k = 2$, $k = 5$, and $k = 7$ in Figure 2 shows an unexpected distinct bimodal pattern: one group of similarity values, e.g. for $k = 2$ around $J \approx 0.6$ and another cluster near $J \approx 0.9$. Accordingly, the CDF in Figure 3 for the blue curve ($k = 2$) rises slowly near 0.6 and then shows a steep jump around 0.9. This indicates that the algorithm consistently discovers two different possible two-cluster partitions in the data. Depending on the random initialization or subsample composition, the k -means procedure either converges to one dominant global split (high similarity, $J \approx 0.9$) or to an alternative boundary that divides the points differently (moderate similarity, $J \approx 0.6$). Similar at other k . The mixture of these outcomes produces two modes in the similarity distribution.

In the context of the linguistic variations across the U.S. this could be a north-south, but also east-west cluster which results in the bimodal behavior. In fact, Figure 5 shows clusterings for $k = 2$ and with different seed. In Figure 5a the north-east (Maine area) belongs to 1, whereas with a different seed the area belongs to cluster 0, which indicates that this area actually forms a third cluster, that is visible when increasing to $k = 3$, cf. Figure 5c.

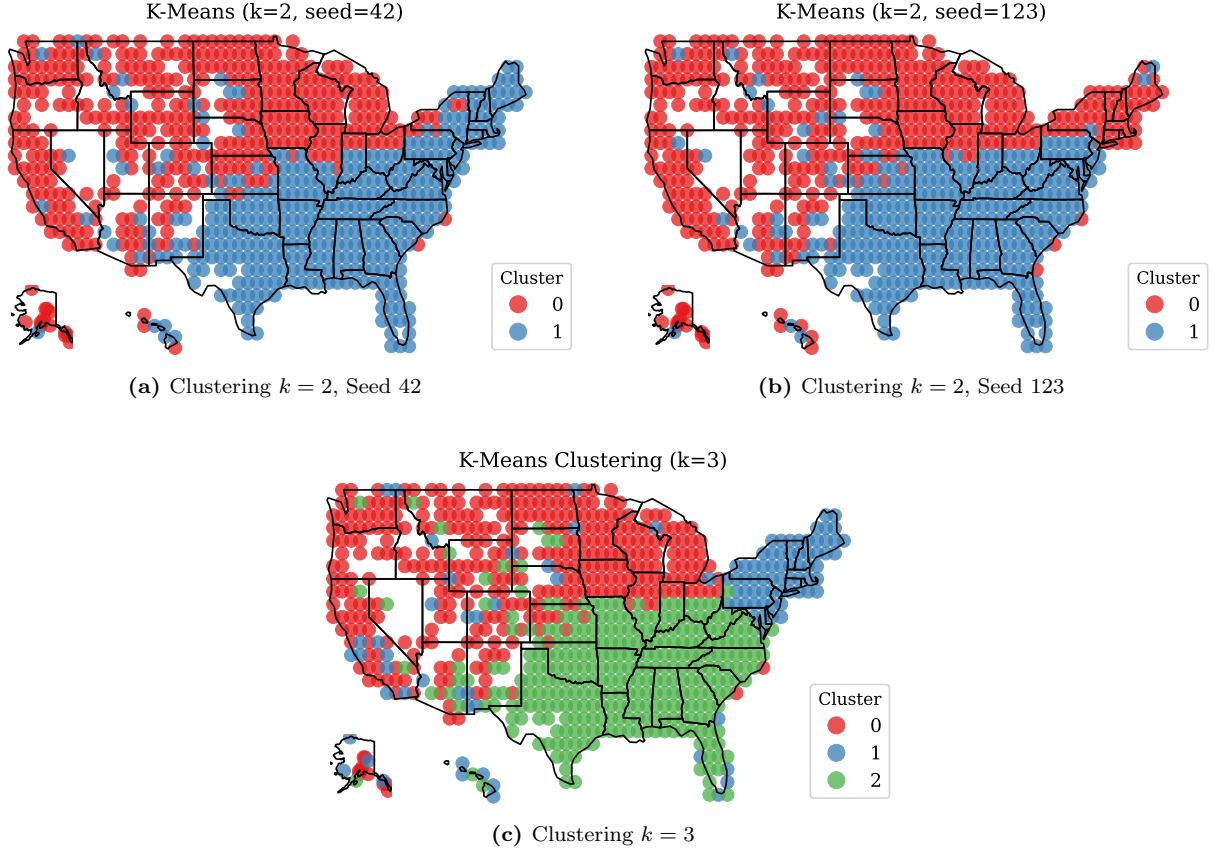


Figure 5: Stability analysis of k-means clustering with $k = 2$. (a) and (b) demonstrate the ambiguous area in the north-east. The kmeans clusters this area almost by 50/50 chance to either clusters. (c) confirms that this area actually forms a third cluster.

Conceptually, this behavior is also consistent with the interpretation of [1]: for very small k values the model is too coarse to capture finer structure, so different subsamples may align with slightly different global separations of the data while still yielding superficially “stable” clusters. The apparent high stability for the upper mode ($J \approx 0.9$) reflects the dominance of one large binary division that many subsamples agree upon, whereas the lower mode around 0.6 corresponds to runs where the second cluster boundary shifts. This split pattern thus suggests that $k = 2$ is *overly coarse* and does not capture a unique, reproducible structure—further supporting the choice of $k = 3$ as the smallest value that provides a single, consistently stable clustering solution.

Choice of k . Based on the distributions (Fig. 2), CDF shifts (Fig. 3), and the stability curve (Fig. 4), the defensible choice for this dataset is $k = 3$ and possibly $k = 4$ or $k = 6$. This avoids the bimodal distributions, as shown for $k = 2$, while remaining in the earliest plateau/shoulder before the pronounced drop for larger k .

Do we trust this method? The method measures *reproducibility under perturbation*, which is a desirable property for clustering and is label-invariant by construction (pairwise co-membership). However, high stability does not guarantee correctness: overly coarse solutions (e.g., $k = 2$) can be stably wrong, and results depend on the subsample fraction m , the clustering algorithm/distance (Euclidean k -means on binary data may be suboptimal), and cluster imbalance. Thus, stability is a strong primary signal but should be confirmed with a complementary criterion.

4.1 Alternative methods for choosing the number of clusters

Several classical approaches have been proposed to determine the optimal number of clusters k in k -means clustering. The most frequently used techniques are the Elbow method, the Silhouette method, and the Gap statistic, cf. [2], [3].

Elbow method. The Elbow method evaluates the within-cluster sum of squares (WCSS) defined as the total squared distance between data points and their assigned cluster centroids. As k increases, WCSS decreases monotonically because more clusters improve the fit. The value of k is chosen at the “elbow”—the point at which further increasing k yields only marginal improvement in WCSS [4]. While intuitive, the elbow is often ambiguous in real data and is based on Euclidean distance, making it less meaningful for binary data or non-Euclidean similarity measures such as Jaccard. Hence, it provides only a rough heuristic for the current analysis.

Silhouette method. The Silhouette method [2] assesses the quality of a clustering by computing, for each observation, the silhouette coefficient

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}},$$

where $a(i)$ is the mean distance between a point and others in its own cluster, and $b(i)$ is the minimum mean distance to points in any other cluster. The average silhouette over all points measures how well the data are separated, and the k that maximizes this average is selected. While this method directly balances cluster cohesion and separation, it relies on the choice of a suitable distance metric. For binary features, Euclidean distance is not ideal, and a Hamming or Jaccard distance would be more appropriate. Thus, the silhouette score can provide a complementary check but not a standalone criterion in this setting.

Gap statistic. The Gap statistic [3] compares the observed within-cluster variation W_k to its expected value under a reference null distribution:

$$\text{Gap}(k) = E^*[\log(W_k^*)] - \log(W_k),$$

where E^* denotes the expectation computed from datasets generated under the null model (typically a uniform or unstructured distribution). The optimal number of clusters is the smallest k for which $\text{Gap}(k)$ is within one standard deviation of the maximum. This approach is statistically principled and accounts for random structure in the data; however, defining an appropriate null model for binary attributes is non-trivial, and the method is computationally demanding because it requires clustering many simulated datasets.

Assessment for this study All three methods provide useful heuristics for continuous data, but they are less well aligned with the binary co-membership stability approach. The Elbow method is highly heuristic, the Silhouette method depends on a suitable distance metric, and the Gap statistic is computationally intensive and challenging to apply meaningfully for binary features. In contrast, the stability-based criterion of Ben-Hur *et al.* [1] directly measures the reproducibility of cluster assignments under subsampling - a property that is both label-invariant and conceptually appropriate for this dataset.

4.2 Summary

In this lab we implemented and analyzed stability-based clustering following Ben-Hur *et al.* [1]. Three similarity measures (Jaccard, Matching, and Correlation) were explained in multiple algorithmic modes, including a direct *normal* $\mathcal{O}(q^2)$ implementation, a memory-efficient *numba*-compiled version, and a sub-quadratic *fast* counting method. The *fast* approach was shown to achieve identical results to the pairwise baseline while reducing both runtime and memory requirements from quadratic to linear order. The similarity functions were then integrated into the stability analysis algorithm to estimate the reproducibility of clusterings under subsampling on the `lingBinary.Rdata` dataset. The resulting stability curves exhibited the expected pattern of high stability at small k and decreasing stability as k increased. Based on these results, $k = 3$ was selected

as the most appropriate number of clusters, balancing robustness and granularity. The lab demonstrated the trade-offs between computational efficiency, statistical stability, and interpretability, and highlighted how implementation choices and parameter settings influence the practical feasibility of stability-based clustering.

5 Academic Honesty

5.1 Statement

I affirm that this report represents my own work. All sources of information, data, figures, and code that are not my own have been properly cited and acknowledged. No unauthorized assistance was received in the completion of this assignment, and I have abided by the principles of academic integrity as required by the course and the institution.

5.2 LLM Usage

An LLM (Large Language Model) was used to assist with this lab. In particular, it was helpful for writing the fast implementation and refining Python code for optimizing with numba, joblib, etc., visualization, and clustering, as well as for improving the structure and clarity of the written report. All analyses and interpretations were ultimately carried out and verified by me, but the LLM provided useful support in streamlining coding tasks and polishing explanations.

5.3 Collaborators

List your collaborators here.

6 Bibliography

References

- [1] A. Ben-Hur, A. Elisseeff, and I. Guyon, “A stability based method for discovering structure in clustered data,” en, in *Biocomputing 2002*, Kauai, Hawaii, USA: WORLD SCIENTIFIC, Dec. 2001, pp. 6–17, ISBN: 978-981-02-4777-5 978-981-279-962-3. DOI: 10.1142/9789812799623_0002. Accessed: Oct. 12, 2025. [Online]. Available: http://www.worldscientific.com/doi/abs/10.1142/9789812799623_0002.
- [2] L. Kaufman and P. J. Rousseeuw, *Finding groups in data: an introduction to cluster analysis* (Wiley series in probability and mathematical statistics), en. Hoboken, N.J: Wiley, 2005, ISBN: 978-0-471-73578-6.
- [3] R. Tibshirani, G. Walther, and T. Hastie, “Estimating the Number of Clusters in a Data Set via the Gap Statistic,” en, *Journal of the Royal Statistical Society. Series B (Statistical Methodology)*, vol. 63, no. 2, pp. 411–423, 2001. [Online]. Available: <http://www.jstor.org/stable/2680607>.
- [4] R. L. Thorndike, “Who Belongs in the Family?” en, *Psychometrika*, vol. 18, no. 4, pp. 267–276, Dec. 1953, ISSN: 0033-3123, 1860-0980. DOI: 10.1007/BF02289263. Accessed: Oct. 15, 2025. [Online]. Available: https://www.cambridge.org/core/product/identifier/S0033312300048730/type/journal_article.